PHYB57 · CHAPTER 20 · 第 20 章

数值微分 · 全面复习讲义

Numerical Differentiation · Comprehensive Review Notes

中英双语 · KaTeX 公式 · 深色阅读版

Bilingual · KaTeX formulas · dark reading edition

KaTeX 公式 · KaTeX | 中英对照 · Bilingual | 每页考点速记 · Key points on every page | 按幻灯片顺序 · Slide order | 拓展 + 6 道练习 · Extensions + 6 problems

覆盖 C20 全部 10 页幻灯片：数值网格、前向/后向/中心差分、Taylor 误差分析、Python 实现与步长减半实验、二阶导数公式、噪声对导数的放大。

> Covers all 10 slides of C20: the numerical grid, forward/backward/central differences, Taylor error analysis, the Python implementation and step-halving experiment, the second-derivative formula, and how noise is amplified by differentiation.

另加幻灯片没有展开的内容（标 拓展 EXT）：中心差分的误差推导、舍入误差与最优步长、噪声–步长权衡、去噪方法；最后是公式总表和 6 道带完整解答的练习题。所有数值都用 Python 复算过。

> Added material the slides do not develop (tagged 拓展 EXT): the central-difference error derivation, round-off error and the optimal step, the noise–step trade-off and denoising methods; then a formula sheet and 6 fully worked practice problems. Every number was recomputed in Python.

PHYB57 复习用 · 2026 年 11 月 5 日

> For PHYB57 review · 5 November 2026

考点速记 KEY POINTS

- 前向/后向差分：一阶，误差 $O(h)$。
  Forward/backward: first order, error $O(h)$.
- 中心差分 $\frac{f_{j+1}-f_{j-1}}{2h}$：二阶 $O(h^2)$。
  Central $\frac{f_{i+1}-f_{i-1}}{2h}$: second order $O(h^2)$.
- $f'' \approx \frac{f_{i+1}-2f_i+f_{i-1}}{h^2}$，误差 $O(h^2)$。
  $f'' \approx \frac{f_{i+1}-2f_i+f_{i-1}}{h^2}$, error $O(h^2)$.
- 噪声 $\epsilon \sin \omega x$ 在 k 阶导数中放大为 $\epsilon\omega^k$。
  Noise $\epsilon \sin \omega x$ grows to $\epsilon\omega^k$ in the k-th derivative.

本章地图与记号

Chapter map and notation

节 Sec.	内容 Content	幻灯片 Slides
1	问题概述：数值网格 · Problem overview: the numerical grid	1
2	三种有限差分 · Three finite differences	2
3	Taylor 展开与精度阶 · Taylor series and order of accuracy +拓展 EXT	3
4	Python 前向差分 · Forward difference in Python	4
5	误差分析：步长减半 · Error analysis: halving h +舍入误差 round-off	5
6	二阶导数的三点公式 · Three-point second derivative	6
7	带噪声的数值微分 · Differentiation with noise +定量分析 quantitative	7–9
8	处理噪声 · Handling noise +去噪方法 remedies	10
—	公式总表 · 练习 Q1–Q6 · 答案与来源 · Formula sheet · Problems · Key & sources	—

记号约定 / Notation

网格与步长 / grid and step

$$x_j = x_0 + j\,h, \qquad f_j = f(x_j)$$

差分算子 / difference operators

$$D_h^+ f_j = \frac{f_{j+1} - f_j}{h}, \quad D_h^- f_j = \frac{f_j - f_{j-1}}{h}, \quad D_h^0 f_j = \frac{f_{j+1} - f_{j-1}}{2h}$$

- p 阶精度：误差 $E(h) \le Ch^p$（$h \to 0$），记作 $O(h^p)$；p 越大，h 变小时误差降得越快。
  > Order p: the error satisfies $E(h) \le Ch^p$ as $h \to 0$, written $O(h^p)$; larger p means the error falls faster as h shrinks.
- 截断误差（truncation error）：截掉 Taylor 级数高阶项造成的误差；舍入误差（round-off）：浮点运算造成的误差。
  > Truncation error: the error from cutting the Taylor series; round-off error: the error from floating-point arithmetic.
- 本讲义中"误差首项"一律按 $f' - D_h f$（精确值减近似值）书写。
  > Throughout, a "leading error term" means $f' - D_h f$ (exact minus approximation).
- 标 拓展 EXT 的内容超出幻灯片，用于理解和应对综合题。
  > Items tagged 拓展 EXT go beyond the slides, for understanding and harder questions.

考点速记 KEY POINTS

- 网格等距：$x_{j+1} - x_j = h$。
  Uniform grid: $x_{j+1} - x_j = h$.
- $O(h^p)$：h 减半，误差约乘 2^{-p}。
  $O(h^p)$: halving h multiplies the error by about 2^{-p}.
- 截断误差随 $h \downarrow$ 减小，舍入/噪声误差随 $h \downarrow$ 增大。
  Truncation falls as $h \downarrow$; round-off and noise errors grow.
- 误差首项按"精确 − 近似"记。
  Leading error = exact − approximation.

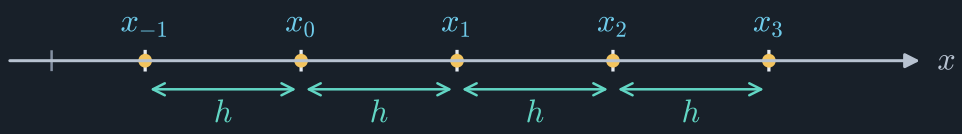
1 数值微分：问题概述

1 Numerical differentiation: problem overview

概念 / Concept

数值微分用一组离散点上的函数值来估计函数的导数，这组点通常称为数值网格（numerical grid）。

> Numerical differentiation estimates the derivative of a function from its values on a discrete set of points, often referred to as a numerical grid.



等距网格：相邻点间距为 h / uniform grid with spacing h

数值网格 / Numerical grid

- 网格是在函数定义域（自变量）上取的一组等距点。
  > A grid is a set of evenly spaced points over the domain of a function (the independent variable).
- 相邻两点的间距记作 h，每个点记作 x_j，j 是下标。
  > The spacing between adjacent points is denoted h, and each point is labelled x_j, where j is the index.

$$x_j = x_0 + j\,h, \qquad h = x_{j+1} - x_j, \qquad N \approx \frac{b-a}{h} \text{ points on } [a, b)$$

为什么重要 / Importance

- 函数只在离散点上已知时很有用，例如传感器数据（如温度测量）。
  > Useful when the function is only known at discrete points, such as sensor data (e.g. temperature measurements).
- 即使函数在现实中是连续的，离散点也能帮助近似它的导数。
  > Even though the function may be continuous in reality, the discrete points help approximate its derivative.

应用 / Application

数值微分在工程和科学中分析真实数据时至关重要：连续的函数往往只能用一组离散观测值来近似。

> Numerical differentiation is critical in engineering and science for analysing real-world data, where continuous functions are often approximated by discrete sets of observations.

考试提醒 / Exam reminder

数值微分只用网格上的数 f_j，不需要 f 的解析式。精度由两样东西决定：步长 h 和 f 的光滑程度（高阶导数的大小）。Python 中 np.arange(0, 2*np.pi, h) 不包含终点 2π。

> Numerical differentiation uses only the grid values f_j, never a formula for f. Accuracy depends on two things: the step h and the smoothness of f (the size of its higher derivatives). In Python, np.arange(0, 2*np.pi, h) excludes the endpoint 2π.

考点速记 KEY POINTS

- 网格点 $x_j = x_0 + jh$，间距 h。
  Grid points $x_j = x_0 + jh$, spacing h.
- 精度取决于 h 与 f 的高阶导数。
  Accuracy depends on h and f's higher derivatives.
- 只有离散数据（测量值）时用数值微分。
  Use it when only discrete (measured) data exist.
- np.arange 不含终点。
  np.arange excludes the endpoint.

2 有限差分近似

2 Finite difference approximations

有限差分法：一种近似函数导数的数值方法。目标：用相邻点上的函数值估计 $f(x)$ 在某点的斜率。

> Finite difference method: a numerical technique to approximate derivatives of functions. Objective: estimate the slope of $f(x)$ at a point using values from neighbouring points.

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

去掉极限、保留有限的 h，就得到差分公式 / drop the limit, keep a finite h: a difference formula

三种基本公式 / Key formulations

1 前向差分 / forward

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_j)}{x_{j+1} - x_j}$$

用 $f(x_j), f(x_{j+1})$ / uses $f(x_j), f(x_{j+1})$

2 后向差分 / backward

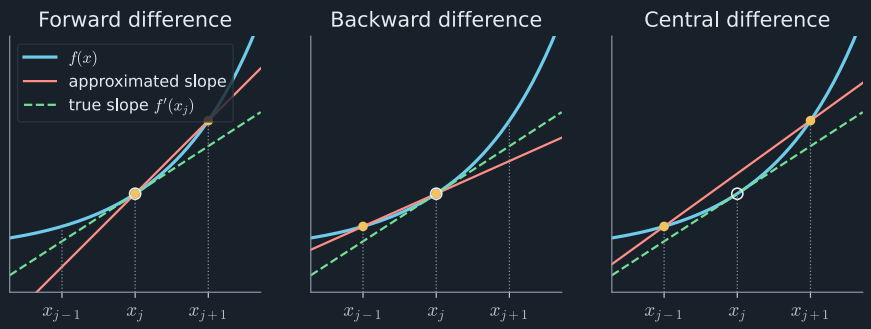
$$f'(x_j) \approx \frac{f(x_j) - f(x_{j-1})}{x_j - x_{j-1}}$$

用 $f(x_j), f(x_{j-1})$ / uses $f(x_j), f(x_{j-1})$

3 中心差分 / central

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_{j-1})}{2h}$$

用 $f(x_{j+1}), f(x_{j-1})$ / uses $f(x_{j+1}), f(x_{j-1})$



红线：差分（割线）斜率；绿虚线：真实切线斜率 $f'(x_j)$ / red: difference (secant) slope; green dashed: true tangent slope $f'(x_j)$

看图：前向和后向割线各偏向一侧；中心割线连接 x_{j-1} 与 x_{j+1}，几乎与切线平行，所以中心差分更准（第 3 节给出原因）。

> From the figure: the forward and backward secants each lean to one side; the central secant joins x_{j-1} and x_{j+1} and is almost parallel to the tangent, so the central difference is more accurate (Section 3 shows why).

注意 / Caution

中心差分的分母是 $2h$（两点相距 $2h$），而且完全不用 $f(x_j)$。端点：前向差分不能用于最后一点，后向差分不能用于第一点，中心差分两端都不能用；N 个点只能得到 $N-1$（单侧）或 $N-2$（中心）个导数值。

> The central difference divides by $2h$ (the two points are $2h$ apart) and never uses $f(x_j)$. At the ends: forward fails at the last point, backward at the first, central at both; N points give only $N-1$ (one-sided) or $N-2$ (central) derivative values.

考点速记 KEY POINTS

- 前向 $\frac{f_{j+1}-f_j}{h}$，后向 $\frac{f_j-f_{j-1}}{h}$。
  Forward $\frac{f_{j+1}-f_j}{h}$, backward $\frac{f_j-f_{j-1}}{h}$.
- 中心差分不用 f_j。
  The central difference skips f_j.
- 中心 $\frac{f_{j+1}-f_{j-1}}{2h}$：分母 $2h$！
  Central $\frac{f_{j+1}-f_{j-1}}{2h}$: divide by $2h$!
- N 点 → $N-1$ 或 $N-2$ 个导数值。
  N points → $N-1$ or $N-2$ derivatives.

3 有限差分方法的精度

3 Accuracy of finite difference methods

精度阶 / Order of accuracy

- 前向与后向差分：一阶精度 $O(h)$，即 $h \to 0$ 时误差线性减小。
  > Forward and backward difference: first-order accuracy, $O(h)$, meaning the error decreases linearly as $h \to 0$.
- 中心差分：二阶精度 $O(h^2)$，即 $h \to 0$ 时误差按平方减小。
  > Central difference: second-order accuracy, $O(h^2)$, meaning the error decreases quadratically as $h \to 0$.

由 Taylor 展开推出前向差分 / Forward difference from the Taylor series

把 $f(x_{j+1})$ 在 x_j 处展开（幻灯片原式）：

> Expand $f(x_{j+1})$ about x_j (the slide's form):

$$\begin{aligned} f(x_{j+1}) &= \frac{f(x_j)(x_{j+1}-x_j)^0}{0!} + \frac{f'(x_j)(x_{j+1}-x_j)^1}{1!} + \frac{f''(x_j)(x_{j+1}-x_j)^2}{2!} + \frac{f'''(x_j)(x_{j+1}-x_j)^3}{3!} + \cdots \\ &= f(x_j) + hf'(x_j) + \frac{h^2}{2!}f''(x_j) + \frac{h^3}{3!}f'''(x_j) + \cdots \qquad (x_{j+1}-x_j = h) \end{aligned}$$

移项并除以 h，解出 $f'(x_j)$：

> Rearrange and divide by h to solve for $f'(x_j)$:

$$f'(x_j) = \frac{f(x_{j+1})-f(x_j)}{h} + \left(-\frac{f''(x_j)h}{2!} - \frac{f'''(x_j)h^2}{3!} - \cdots\right)$$

$$f'(x_j) = \frac{f(x_{j+1})-f(x_j)}{h} + O(h)$$

括号里的就是截断误差，首项 $-\frac{h}{2}f''(x_j)$ 正比于 h / the bracket is the truncation error; its leading term $-\frac{h}{2}f''(x_j)$ is proportional to h

后向差分：把 h 换成 $-h$ / Backward difference: replace h by $-h$

$$f(x_{j-1}) = f(x_j) - hf'(x_j) + \frac{h^2}{2}f''(x_j) - \cdots \;\Rightarrow\; f'(x_j) = \frac{f(x_j)-f(x_{j-1})}{h} + \frac{h}{2}f''(x_j) + O(h^2)$$

高阶项 / Higher-order terms

高阶项来自 Taylor 展开，可用来提高近似精度。Taylor 展开说明了截断误差从何而来，也说明了怎样利用高阶项（组合更多网格点、消去低阶误差）提高精度。

> Higher-order terms come from the Taylor expansion and are used to improve accuracy. The Taylor series shows how truncation errors arise, and how higher-order terms (combining more grid points to cancel low-order errors) improve precision.

考试提醒 / Exam reminder

前向和后向的误差首项 $\mp\frac{h}{2}f''$ 大小相同、符号相反，两者取平均就消去了 $O(h)$ 项：$\frac{1}{2}(D_h^+ + D_h^-) = D_h^0$，即中心差分（下一页）。

> The forward and backward leading errors $\mp\frac{h}{2}f''$ are equal and opposite, so their average cancels the $O(h)$ term: $\frac{1}{2}(D_h^+ + D_h^-) = D_h^0$, the central difference (next page).

考点速记 KEY POINTS

- 前向：$f' = D_h^+ f - \frac{h}{2}f'' + \cdots$。
  Forward: $f' = D_h^+ f - \frac{h}{2}f'' + \cdots$.
- 后向：$f' = D_h^- f + \frac{h}{2}f'' + \cdots$。
  Backward: $f' = D_h^- f + \frac{h}{2}f'' + \cdots$.
- 除以 h 使误差降一阶：$O(h^2)/h = O(h)$。
  Dividing by h drops one order: $O(h^2)/h = O(h)$.
- 前后平均 = 中心差分。
  Average of forward and backward = central.

3.2 中心差分为什么是二阶 拓展 EXT

3.2 Why the central difference is second order

幻灯片只给出结论 $O(h^2)$。推导：把 $f(x_{j+1})$ 与 $f(x_{j-1})$ 的展开相减，偶次项（f、f''、…）抵消，只剩奇次项。

> The slide states only the result $O(h^2)$. Derivation: subtract the expansions of $f(x_{j+1})$ and $f(x_{j-1})$; the even terms (f, f'', …) cancel and only odd terms remain.

$$f(x_{j+1}) - f(x_{j-1}) = 2hf'(x_j) + \frac{2h^3}{3!}f'''(x_j) + \frac{2h^5}{5!}f^{(5)}(x_j) + \cdots$$

$$\Rightarrow \quad f'(x_j) = \frac{f(x_{j+1}) - f(x_{j-1})}{2h} - \frac{h^2}{6}f'''(x_j) + O(h^4)$$

"阶"在数值上意味着什么 / What the order means in numbers

若 $E(h) \approx Ch^p$，则 $\log E = \log C + p\log h$：在双对数（log-log）图上是一条斜率为 p 的直线。

> If $E(h) \approx Ch^p$, then $\log E = \log C + p\log h$: a straight line of slope p on a log-log plot.

步长减半 / halving h

$$\frac{E(h/2)}{E(h)} \approx 2^{-p}$$

$p = 1$：误差减半；$p = 2$：变为 1/4

/ $p = 1$: halves; $p = 2$: quarters

由数据测阶 / measured order

$$p \approx \log_2 \frac{E(h)}{E(h/2)} = \frac{\Delta \log E}{\Delta \log h}$$

即 log-log 图的斜率 / the slope of the log-log plot

方法 Method	所用点 Points	误差首项 $f' - D$ Leading error	阶 Order	$h \to h/2$
前向 Forward	$x_j,\ x_{j+1}$	$-\frac{h}{2}f''(x_j)$	1	×1/2
后向 Backward	$x_{j-1},\ x_j$	$+\frac{h}{2}f''(x_j)$	1	×1/2
中心 Central	$x_{j-1},\ x_{j+1}$	$-\frac{h^2}{6}f'''(x_j)$	2	×1/4
三点 f'' · 3-point f''	x_{j-1}, x_j, x_{j+1}	$-\frac{h^2}{12}f^{(4)}(x_j)$	2	×1/4

考试提醒 / Exam reminder

误差首项里出现哪一阶导数也要会说：一阶差分的误差取决于 f''，中心差分取决于 f'''。所以对线性函数前向差分精确；对二次函数中心差分精确（$f''' = 0$）。

> Know which derivative sets the error: one-sided differences depend on f'', the central one on f'''. So forward differences are exact for linear functions, and central differences are exact for quadratics ($f''' = 0$).

考点速记 KEY POINTS

- 相减消去偶次项 → 中心差分 $O(h^2)$。
  Subtracting cancels even terms → central is $O(h^2)$.
- log-log 图斜率 = 阶数 p。
  Log-log slope = order p.
- 中心误差首项 $-\frac{h^2}{6}f'''$。
  Central leading error $-\frac{h^2}{6}f'''$.
- $p \approx \log_2[E(h)/E(h/2)]$。
  $p \approx \log_2[E(h)/E(h/2)]$.

4 用 Python 做有限差分近似

4 Using Python for finite difference approximation

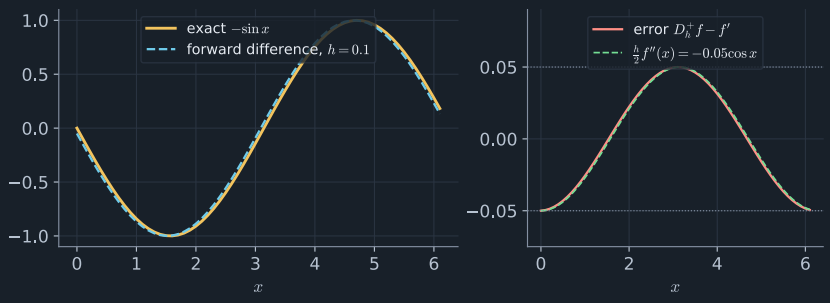
- 用 numpy 库计算数值导数。示例函数 $f(x)=\cos x$，已知导数 $f'(x)=-\sin x$。
  > We compute numerical derivatives with numpy. Example function $f(x)=\cos x$, with known derivative $f'(x)=-\sin x$.
- 对它用前向差分，并把结果与精确解比较。
  > The forward difference is applied and the result is compared with the exact solution.

```python
import numpy as np
import matplotlib.pyplot as plt

h = 0.1
x = np.arange(0, 2*np.pi, h)
y = np.cos(x)
forward_diff = np.diff(y) / h
x_diff = x[:-1]
exact_solution = -np.sin(x_diff)

plt.plot(x_diff, forward_diff, '--', label='Approximation')
plt.plot(x_diff, exact_solution, label='Exact')
plt.legend()
plt.show()
```

- x 有 63 个点（0 到 6.2，不含 2π）。
  > x has 63 points (0 to 6.2; 2π excluded).
- np.diff(y) 即 y[1:] − y[:-1]，长度 62 = $N-1$。
  > np.diff(y) is y[1:] − y[:-1], length 62 = $N-1$.
- 除以 h 得前向差分；它属于左端点 x_j，所以配 x[:-1]（去掉最后一点）。
  > Dividing by h gives the forward difference; it belongs to the left point x_j, so pair it with x[:-1] (drop the last point).



左：近似与精确解；右：误差 $D_h^+ f - f' \approx \frac{h}{2} f'' = -0.05\cos x$ / left: approximation vs exact; right: error $D_h^+ f - f' \approx \frac{h}{2} f'' = -0.05\cos x$

最大误差为 0.0500，与理论 $\frac{h}{2}\max|f''| = \frac{0.1}{2}\times 1 = 0.05$ 一致（第 5 节的出发点）。

> The maximum error is 0.0500, matching the theory $\frac{h}{2}\max|f''| = \frac{0.1}{2}\times 1 = 0.05$ (the starting point of Section 5).

看图要点 / Reading the plot

虚线像是整体向左平移了 $h/2$：$\frac{f(x_{j+1})-f(x_j)}{h}$ 其实是中点 $x_j+\frac{h}{2}$ 处的中心差分（二阶精度）。若画在 x[:-1] + h/2，最大误差从 0.050 降到 0.0004。

> The dashed curve looks shifted left by $h/2$: $\frac{f(x_{i+1})-f(x_i)}{h}$ is really a central difference at the midpoint $x_j+\frac{h}{2}$ (second order). Plotted at x[:-1] + h/2, the maximum error drops from 0.050 to 0.0004.

考点速记 KEY POINTS

- np.diff 长度为 $N-1$。
  np.diff has length $N-1$.
- $h=0.1$：最大误差 $\approx 0.05 = \frac{h}{2}\max|f''|$。
  $h=0.1$: max error $\approx 0.05 = \frac{h}{2}\max|f''|$.
- 前向差分配 x[:-1]。
  Forward difference pairs with x[:-1].
- 前向差分在中点处是二阶的。
  At the midpoint the forward difference is second order.

5 误差分析：步长减半实验

5 Error analysis: halving the step size

上一页图中数值解与精确解的最大误差约为 0.05；步长 h 减小时，误差线性减小。

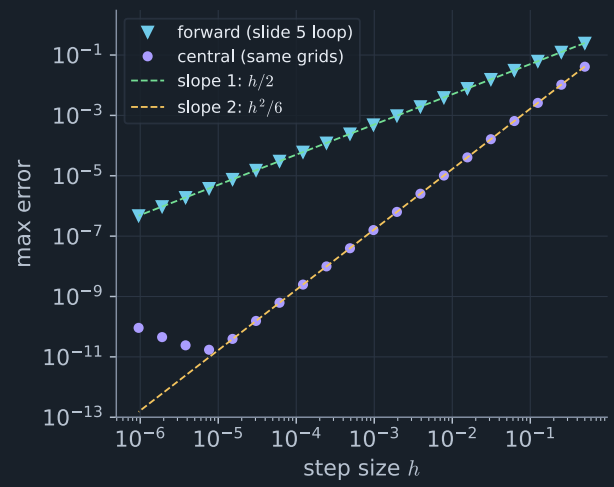
> The maximum error between the numerical and exact solutions in the previous figure is about 0.05, and it decreases linearly as the step size h is reduced.

```python
# define step size
h = 1
# define number of iterations to perform
iterations = 20
# list to store our step sizes
step_size = []
# list to store max error for each step size
max_error = []

for i in range(iterations):
    # halve the step size
    h /= 2
    # store this step size
    step_size.append(h)
    # compute new grid
    x = np.arange(0, 2 * np.pi, h)
    # compute function value at grid
    y = np.cos(x)
    # compute vector of forward differences
    forward_diff = np.diff(y)/h
    # compute corresponding grid
    x_diff = x[:-1]
    # compute exact solution
    exact_solution = -np.sin(x_diff)

    # Compute max error between
    # numerical derivative and exact solution
    max_error.append(
        max(abs(exact_solution - forward_diff)))

# produce log-log plot of max error
# versus step size
plt.figure(figsize = (12, 8))
plt.loglog(step_size, max_error, 'v')
plt.show()
```



中心差分在 $h < 10^{-5}$ 后被舍入误差抬高（见 5.2）

below $h \approx 10^{-5}$ round-off lifts the central error (see 5.2)

- 循环把 h 减半 20 次，从 0.5 到 9.5×10^{-7}。
  > The loop halves h 20 times, from 0.5 to 9.5×10^{-7}.
- 蓝色三角在 log-log 图上成直线，拟合斜率 $0.9998 \approx 1$：一阶。
  > The blue triangles form a line of fitted slope $0.9998 \approx 1$: first order.
- 误差 $\approx h/2$：$h = 9.5 \times 10^{-7}$ 时误差 4.8×10^{-7}。
  > Error $\approx h/2$: at $h = 9.5 \times 10^{-7}$ the error is 4.8×10^{-7}.
- 紫色圆点（拓展）：同样网格上的中心差分，斜率 2。
  > Purple dots (extension): central difference on the same grids, slope 2.

考试提醒 / Exam reminder

读 log-log 图：横轴每降一个数量级，一阶方法误差降一个数量级，二阶方法降两个。题目给出两组 (h, E) 时，用 $p = \Delta \log E / \Delta \log h$ 判断阶数。

> Reading a log-log plot: per decade of h, a first-order error falls one decade and a second-order error two. Given two (h, E) pairs, find the order with $p = \Delta \log E / \Delta \log h$.

考点速记 KEY POINTS

- 前向差分 log-log 斜率 1。
  Forward difference: log-log slope 1.
- 中心差分斜率 2，同样 h 下误差小得多。
  Central: slope 2, far smaller at the same h.
- $\cos x$ 的最大误差 $\approx h/2$。
  Max error for $\cos x \approx h/2$.
- plt.loglog 画双对数图。
  plt.loglog draws the log-log plot.

5.2 h 不能无限小：舍入误差 拓展 EXT

5.2 h cannot go to zero: round-off error

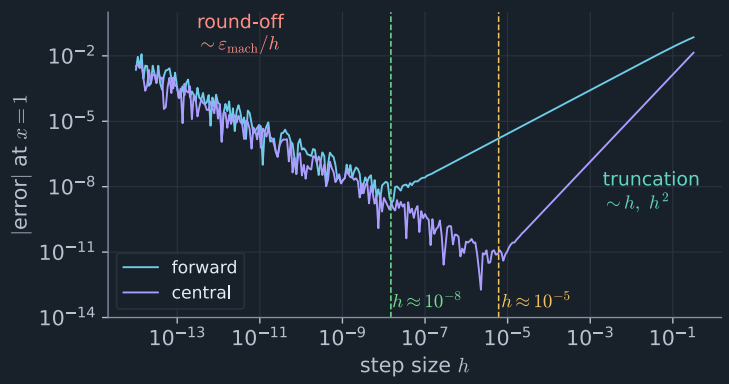
幻灯片只算到 $h \approx 10^{-6}$。继续减小 h 时，$f(x_{j+1}) - f(x_j)$ 是两个几乎相等的数相减（相消误差，catastrophic cancellation）：每个函数值带有约 $\varepsilon_{\text{mach}}|f|$ 的浮点误差，除以 h 后被放大。

> The slide stops near $h \approx 10^{-6}$. For smaller h, $f(x_{j+1}) - f(x_j)$ subtracts two nearly equal numbers (catastrophic cancellation): each value carries a floating-point error of about $\varepsilon_{\text{mach}}|f|$, and dividing by h amplifies it.

$$E(h) \approx \underbrace{\frac{h}{2}|f''|}_{\text{truncation}} + \underbrace{\frac{2\varepsilon_{\text{mach}}|f|}{h}}_{\text{round-off}}, \qquad h^* = 2\sqrt{\varepsilon_{\text{mach}}\frac{|f|}{|f''|}} \sim 10^{-8}$$

双精度 $\varepsilon_{\text{mach}} = 2.2 \times 10^{-16}$；中心差分 $h^* \sim \varepsilon_{\text{mach}}^{1/3} \sim 10^{-5}$

/ double precision $\varepsilon_{\text{mach}} = 2.2 \times 10^{-16}$; central: $h^* \sim \varepsilon_{\text{mach}}^{1/3} \sim 10^{-5}$



$\cos x$ 在 $x = 1$ 处的误差：V 形曲线，左支斜率 -1（舍入），右支斜率 1 或 2（截断）

/ error for $\cos x$ at $x = 1$: a V shape, left branch slope -1 (round-off), right slope 1 or 2 (truncation)

h	10^{-4}	10^{-6}	10^{-8}	10^{-10}	10^{-12}
前向误差 forward error	2.7×10^{-5}	2.7×10^{-7}	8.1×10^{-9}	3.5×10^{-7}	7.8×10^{-5}

注意 / Caution

"h 越小越准"只在截断误差占主导时成立。$h = 10^{-12}$ 的误差比 $h = 10^{-4}$ 还大。这与第 7 节的噪声问题结构相同：把 $\varepsilon_{\text{mach}}|f|$ 换成测量噪声 δ。

> "Smaller h is more accurate" holds only while truncation dominates. At $h = 10^{-12}$ the error exceeds that at $h = 10^{-4}$. Section 7's noise problem has the same structure, with measurement noise δ in place of $\varepsilon_{\text{mach}}|f|$.

考点速记 KEY POINTS

- 舍入误差 $\sim \varepsilon_{\text{mach}}/h$，随 $h \downarrow$ 增大。
  Round-off $\sim \varepsilon_{\text{mach}}/h$ grows as $h \downarrow$.
- 中心最优 $h^* \sim \varepsilon_{\text{mach}}^{1/3} \approx 10^{-5}$。
  Central optimum $h^* \sim \varepsilon_{\text{mach}}^{1/3} \approx 10^{-5}$.
- 前向最优 $h^* \sim \sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$。
  Forward optimum $h^* \sim \sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$.
- 总误差曲线呈 V 形。
  The total error curve is V-shaped.

6 用 Taylor 级数近似高阶导数

6 Approximating higher-order derivatives using Taylor series

概述 / Concept overview

高阶导数也可以用由 Taylor 展开导出的有限差分来近似。解析解难以得到时，这一技巧在数值分析中必不可少。

> Higher-order derivatives can be approximated with finite differences derived from Taylor series expansions. This technique is essential in numerical analysis when analytical solutions are difficult to obtain.

步骤 1：在 x_{j-1} 和 x_{j+1} 处展开 / Step 1: expand $f(x)$ at x_{j-1} and x_{j+1}

$$f(x_{j-1}) = f(x_j) - hf'(x_j) + \frac{h^2 f''(x_j)}{2} - \frac{h^3 f'''(x_j)}{6} + \frac{h^4 f^{(4)}(x_j)}{24} - \cdots$$

$$f(x_{j+1}) = f(x_j) + hf'(x_j) + \frac{h^2 f''(x_j)}{2} + \frac{h^3 f'''(x_j)}{6} + \frac{h^4 f^{(4)}(x_j)}{24} + \cdots$$

步骤 2：两式相加 / Step 2: add the expansions

$$f(x_{j-1}) + f(x_{j+1}) = 2f(x_j) + h^2 f''(x_j) + \frac{h^4 f^{(4)}(x_j)}{12} + \cdots$$

奇次项（f'、f'''）抵消；两式各贡献 $\frac{h^4}{24} f^{(4)}$ / odd terms (f', f''') cancel; each expansion contributes $\frac{h^4}{24} f^{(4)}$

步骤 3：解出 $f''(x_j)$，二阶精度 / Step 3: solve for $f''(x_j)$, second order

$$f''(x_j) \approx \frac{f(x_{j+1}) - 2f(x_j) + f(x_{j-1})}{h^2}, \qquad f'' - \text{approx} = -\frac{h^2}{12} f^{(4)}(x_j) + \cdots$$

误差项 $O(h^2)$：h 减小，精度提高；权重 $[1, -2, 1]/h^2$

/ error term $O(h^2)$: accuracy improves as h decreases; weights $[1, -2, 1]/h^2$

编者注 / Editorial note

幻灯片（以及它所依据的教材 20.3 节）把相加后的 h^4 项写成 $\frac{h^4 f^{(4)}}{24}$，应为 $\frac{h^4 f^{(4)}}{12}$。这不影响 f'' 公式和 $O(h^2)$ 结论，只影响误差系数。数值验证（练习 Q4）：e^x 在 0 处、$h = 0.1$，误差 $8.34 \times 10^{-4} \approx h^2/12 = 8.33 \times 10^{-4}$。

> The slide (and textbook §20.3 it follows) writes the summed h^4 term as $\frac{h^4 f^{(4)}}{24}$; it should be $\frac{h^4 f^{(4)}}{12}$. The f'' formula and the $O(h^2)$ conclusion are unaffected; only the error coefficient changes. Numerical check (Q4): e^x at 0 with $h = 0.1$ gives error $8.34 \times 10^{-4} \approx h^2/12 = 8.33 \times 10^{-4}$.

应用 / Applications

用于物理和工程中的波动方程、流体力学和结构分析，这些模拟都需要高阶导数。numpy 写法：(y[2:] – 2*y[1:-1] + y[:-2]) / h**2，对应 x[1:-1]。

> Used in physics and engineering for wave equations, fluid dynamics and structural analysis, where simulations need higher-order derivatives. In numpy: (y[2:] – 2*y[1:-1] + y[:-2]) / h**2, on x[1:-1].

考点速记 KEY POINTS

- $f'' \approx \frac{f_{j+1} - 2f_j + f_{j-1}}{h^2}$，$O(h^2)$。
  $f'' \approx \frac{f_{j+1} - 2f_j + f_{j-1}}{h^2}$, $O(h^2)$.
- 误差首项 $-\frac{h^2}{12} f^{(4)}$（幻灯片 /24 有误）。
  Leading error $-\frac{h^2}{12} f^{(4)}$ (slide's /24 is a slip).
- 相加消奇次项；相减消偶次项。
  Adding cancels odd terms; subtracting cancels even.
- 分母 h^2，不是 $2h$。
  Denominator h^2, not $2h$.

7 带噪声的数值微分

7 Numerical differentiation with noise

定义 / Definition

- 数值微分常常是用离散数据点来近似函数的导数。
  > Numerical differentiation often involves approximating derivatives of a function given by discrete data points.
- 在实际应用中，这些数据点会因测量不准或外界扰动而被噪声污染。
  > In real-world applications these data points can be contaminated by noise due to measurement inaccuracies or external perturbations.

噪声的影响 / Impact of noise

- 噪声会扭曲函数值，使算出的导数产生误差。
  > Noise can distort function values, leading to errors in the computed derivatives.
- 比喻：在拥挤的房间里听人说话。说话声是函数，背景噪声给信号加上了失真。
  > Example: imagine listening to a conversation in a crowded room. The speech is the function; background noise distorts the signal.

示例 / Illustration

原信号 / clean signal

$$f(x) = \cos(x)$$

加噪声 / with noise

$$f_{\epsilon,\omega}(x) = \cos(x) + \epsilon \sin(\omega x)$$

ϵ 小（如 0.01），ω 大（如 100）

/ ϵ small (e.g. 0.01), ω large (e.g. 100)

噪声项 $\epsilon \sin(\omega x)$ 代表叠加在原信号 $\cos(x)$ 上的高频噪声，它使导数对噪声很敏感，ω 大时尤其如此。

> The noise term $\epsilon \sin(\omega x)$ represents high-frequency noise superimposed on the original signal $\cos(x)$, making the derivative sensitive to noise, particularly when ω is large.

$$\frac{\mathrm{d}}{\mathrm{d}x}\left[\epsilon \sin(\omega x)\right] = \epsilon\,\omega \cos(\omega x) \qquad \text{amplitude}: \ \epsilon \ \to \ \epsilon\omega$$

求导把噪声幅度乘以频率 ω / differentiation multiplies the noise amplitude by its frequency ω

考试提醒 / Exam reminder

判断噪声是否要紧，要看**导数里**的幅度 $\epsilon\omega$，而不是函数里的 ϵ。$\epsilon = 0.01$、$\omega = 100$ 时，函数里 1% 的噪声在导数里变成 100%。

> Judge noise by its amplitude **in the derivative**, $\epsilon\omega$, not by ϵ in the function. With $\epsilon = 0.01$, $\omega = 100$, 1% noise in the function becomes 100% in the derivative.

考点速记 KEY POINTS

- 模型：$f_{\epsilon,\omega} = \cos x + \epsilon \sin \omega x$。
  Model: $f_{\epsilon,\omega} = \cos x + \epsilon \sin \omega x$.
- 噪声：小幅度 ϵ、高频率 ω。
  Noise: small amplitude ϵ, high frequency ω.
- 求导后幅度 $\epsilon \to \epsilon\omega$。
  After differentiating: $\epsilon \to \epsilon\omega$.
- $\epsilon = 0.01$, $\omega = 100$：$\epsilon\omega = 1$。
  $\epsilon = 0.01$, $\omega = 100$: $\epsilon\omega = 1$.

7.2 画出带噪声的函数

7.2 Plotting the noisy function

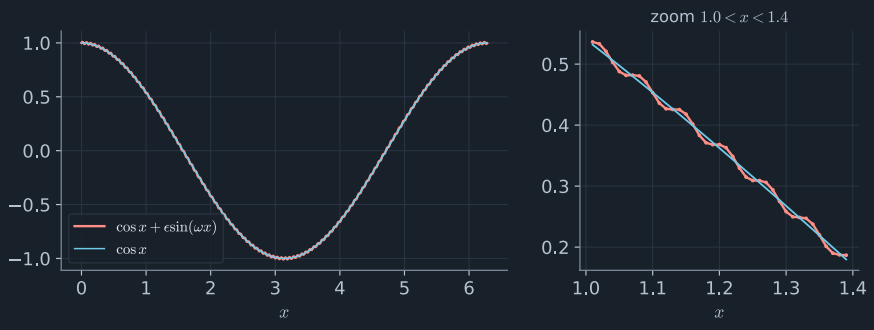
```python
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-poster')
%matplotlib inline

x = np.arange(0, 2*np.pi, 0.01)
# compute function
omega = 100
epsilon = 0.01

y = np.cos(x)
y_noise = y + epsilon*np.sin(omega*x)

# Plot solution
plt.figure(figsize = (12, 8))
plt.plot(x, y_noise, 'r-', \
         label = 'cos(x) + noise')
plt.plot(x, y, 'b-', \
         label = 'cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

- 步长 0.01：在 $[0, 2\pi)$ 上有 629 个点。
  > Step 0.01: 629 points on $[0, 2\pi)$.
- $\omega = 100$：噪声周期 $2\pi/100 \approx 0.063$，每个周期约 6 个网格点，网格刚好能分辨噪声。
  > $\omega = 100$: noise period $2\pi/100 \approx 0.063$, about 6 grid points per cycle, so the grid just resolves the noise.
- $\epsilon = 0.01$ 只有信号幅度的 1%：整体图上两条曲线重合，看不出噪声（幻灯片原图）。
  > $\epsilon = 0.01$ is 1% of the signal amplitude: at full scale the two curves overlap and the noise is invisible (the slide's figure).
- 右下放大图才能看到细小的波纹。
  > Only the zoom (below right) shows the small ripple.



左：全图，红蓝重合；右：$1.0 < x < 1.4$ 放大 / left: full view, red and blue overlap; right: zoom on $1.0 < x < 1.4$

运行代码时注意 / When running this code

新版 Matplotlib（本机 3.10 已验证）中 'seaborn-poster' 会报 OSError，改写为 'seaborn-v0_8-poster'。%matplotlib inline 是 Jupyter 魔法命令，只能在 notebook 里用，放进 .py 脚本会报语法错误。

> In current Matplotlib (checked on 3.10) 'seaborn-poster' raises OSError; write 'seaborn-v0_8-poster' instead. %matplotlib inline is a Jupyter magic command: it works only in a notebook and is a syntax error in a .py script.

考点速记 KEY POINTS

- 函数里的噪声肉眼几乎看不见。
  Noise in the function is nearly invisible.
- "看不见"不等于"导数里不重要"。
  Invisible here ≠ harmless in the derivative.
- 噪声周期 $2\pi/\omega \approx 0.063$。
  Noise period $2\pi/\omega \approx 0.063$.
- 样式名改为 'seaborn-v0_8-poster'。
  Style name is now 'seaborn-v0_8-poster'.

7.3 噪声在导数中被放大

7.3 Noise is amplified in the derivative

原信号的导数 / derivative of the clean signal

$$f'(x) = -\sin(x)$$

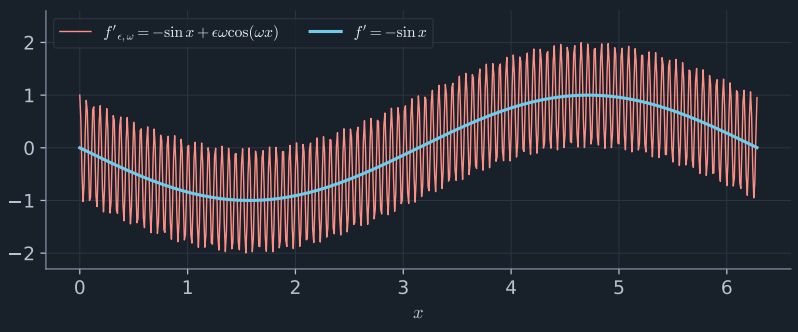
带噪信号的导数 / derivative of the noisy signal

$$f'_{\epsilon,\omega}(x) = -\sin(x) + \epsilon\omega\cos(\omega x)$$

```
x = np.arange(0, 2*np.pi, 0.01)
# compute function
y = -np.sin(x)
y_noise = y + epsilon*omega*np.cos(omega*x)

# Plot solution
plt.figure(figsize = (12, 8))
plt.plot(x, y_noise, 'r-', \
         label = 'Derivative cos(x) + noise')
plt.plot(x, y, 'b-', \
         label = 'Derivative of cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

- 这段代码画的是**解析导数**，不是差分：即使导数算得完全精确，噪声也已占主导。
  > This code plots the **analytic derivative**, not a finite difference: even an exact derivative is dominated by noise.
- 幅度 $\epsilon\omega = 0.01 \times 100 = 1$，与信号 $-\sin x$ 的幅度相同。
  > Amplitude $\epsilon\omega = 0.01 \times 100 = 1$, the same as the signal $-\sin x$.
- 红线在 $-\sin x \pm 1$ 之间振荡，范围 $[-2, 2]$，与幻灯片图一致。
  > The red curve oscillates within $-\sin x \pm 1$, range $[-2, 2]$, as in the slide.



噪声放大了高频分量 $\epsilon\omega\cos(\omega x)$，$\omega$ 越大，与真实导数的偏差越大。每多求一次导就再乘一个 ω：

> The noise amplifies the high-frequency component $\epsilon\omega\cos(\omega x)$, leading to large deviations from the true derivative as ω increases. Each further derivative multiplies by another ω:

$$\left|\frac{\mathrm{d}^k}{\mathrm{d}x^k}\epsilon\sin(\omega x)\right|_{\max} = \epsilon\,\omega^k: \qquad f:\ 0.01, \quad f':\ 1, \quad f'':\ 100$$

考点速记 KEY POINTS

- $f'_{\epsilon,\omega} = -\sin x + \epsilon\omega\cos\omega x$。
  $f'_{\epsilon,\omega} = -\sin x + \epsilon\omega\cos\omega x$.
- 本例 f'' 噪声是信号的 100 倍。
  Here noise in f'' is 100× the signal.
- k 阶导数噪声幅度 $\epsilon\omega^k$。
  Noise in the k-th derivative: $\epsilon\omega^k$.
- 问题出在数据，不在差分公式。
  The problem is the data, not the formula.

7.4 差分如何放大噪声 拓展 EXT

7.4 How finite differences amplify noise

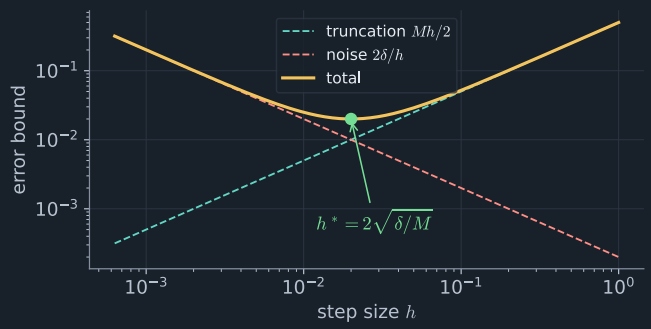
一般的测量噪声 / General measurement noise

设数据 $\tilde f_j = f_j + e_j$，噪声 $|e_j| \le \delta$。前向差分中噪声部分为 $\frac{e_{j+1}-e_j}{h}$，最坏时为 $\frac{2\delta}{h}$：h 越小越大，与截断误差方向相反。

> Let the data be $\tilde f_j = f_j + e_j$ with noise $|e_j| \le \delta$. The noise part of the forward difference is $\frac{e_{j+1}-e_j}{h}$, up to $\frac{2\delta}{h}$ in the worst case: it grows as h shrinks, opposite to the truncation error.

$$\left| D_h^+ \tilde f - f' \right| \le \frac{Mh}{2} + \frac{2\delta}{h}, \quad M = \max|f''| \qquad \Rightarrow \qquad h^* = 2\sqrt{\delta/M}, \quad E_{\min} = 2\sqrt{M\delta}$$

对 h 求导令其为零：$\frac{M}{2} = \frac{2\delta}{h^2}$ / set the h-derivative to zero: $\frac{M}{2} = \frac{2\delta}{h^2}$



$\delta = 10^{-4}$、$M = 1$：$h^* = 0.02$，最小误差 0.02 / $\delta = 10^{-4}$, $M = 1$: $h^* = 0.02$, minimum error 0.02

幻灯片的正弦噪声 / The slide's sinusoidal noise

$$\frac{\epsilon\sin(\omega(x+h)) - \epsilon\sin(\omega x)}{h} = \frac{2\epsilon\sin(\omega h/2)}{h}\cos\left(\omega\left(x+\tfrac{h}{2}\right)\right)$$

幅度 $\frac{2\epsilon|\sin(\omega h/2)|}{h}$。$h = 0.01$ 时 $\omega h = 1$，幅度 $= 0.96 \approx \epsilon\omega = 1$：网格分辨得出噪声，差分几乎把它完整地放大出来。$\omega h \ll 1$ 时幅度趋于 $\epsilon\omega$。

> Amplitude $\frac{2\epsilon|\sin(\omega h/2)|}{h}$. At $h = 0.01$, $\omega h = 1$ and the amplitude is $0.96 \approx \epsilon\omega = 1$: the grid resolves the noise and the difference amplifies almost all of it. For $\omega h \ll 1$ the amplitude tends to $\epsilon\omega$.

考点速记 KEY POINTS

- 噪声误差 $\le 2\delta/h$。
  Noise error $\le 2\delta/h$.
- 最小误差 $2\sqrt{M\delta}$，不会趋于 0。
  Minimum error $2\sqrt{M\delta}$, never zero.
- 最优 $h^* = 2\sqrt{\delta/M}$。
  Optimal $h^* = 2\sqrt{\delta/M}$.
- 正弦噪声：$\frac{2\epsilon|\sin(\omega h/2)|}{h} \to \epsilon\omega$。
  Sinusoid: $\frac{2\epsilon|\sin(\omega h/2)|}{h} \to \epsilon\omega$.

8 处理数值微分中的噪声

8 Handling noise in numerical differentiation

难点 / Challenges

- 噪声在导数中引入高频振荡，使真实信号和噪声难以区分。

  > Noise introduces high-frequency oscillations in the derivative, making it difficult to distinguish the true signal from the noise.

- 高频噪声对导数的影响格外大，因为求导会放大噪声。

  > High-frequency noise has a disproportionate impact on derivative calculations because differentiation amplifies the noise.

结论 / Conclusion

数值微分对噪声高度敏感，高阶导数尤甚。在精度要求高的工程应用中，降低这种敏感性至关重要。

> Numerical differentiation is highly sensitive to noise, especially for higher-order derivatives. Mitigating this sensitivity is crucial in engineering applications where precision is key.

常用对策 拓展 EXT / Common remedies

方法 Method	做法 How	代价 Cost
增大步长 · Larger h	取 $h \approx h^* = 2\sqrt{\delta/M}$ · use $h \approx h^*$	截断误差变大 · more truncation error
先平滑再求导 · Smooth first	滑动平均或低通滤波后再差分 · moving average or low-pass filter, then difference	抹平尖锐特征 · blurs sharp features
Savitzky–Golay	滑动窗口内拟合多项式并对其求导 · fit a polynomial in a sliding window, differentiate the fit · savgol_filter(y, w, k, deriv=1, delta=h)	需选窗口与阶数 · choose window and degree
样条/最小二乘拟合 · Spline / least squares	先拟合光滑函数，再解析求导 · fit a smooth function, differentiate it analytically	需选光滑参数 · smoothing parameter
频域滤波 · Fourier filter	FFT 后去掉高频，再乘 $i\omega$ 求导 · FFT, cut high frequencies, multiply by $i\omega$	适合等距、周期数据 · uniform, periodic data
正则化微分 · Regularised	Tikhonov 或全变差（TV）正则化 · Tikhonov or total-variation regularisation	计算量大 · more computation

注意 / Caution

所有对策都是在"去掉噪声"和"保留真实细节"之间取舍：滤波截止频率必须低于噪声频率、高于信号频率。本例信号频率 1、噪声频率 100，两者分得很开，所以容易处理；频率接近时没有办法完全分开。

> Every remedy trades noise removal against keeping real detail: the filter cutoff must sit below the noise frequency and above the signal's. Here the signal frequency is 1 and the noise 100, well separated and easy to handle; when the two are close they cannot be fully separated.

考点速记 KEY POINTS

- 求导放大高频噪声，高阶导数最敏感。
  Differentiation amplifies high frequencies; higher orders are worst.
- 先平滑/拟合，再求导。
  Smooth or fit first, then differentiate.
- 有噪声时 h 不宜过小。
  With noise, do not make h too small.
- 截止频率介于信号与噪声频率之间。
  Cutoff between signal and noise frequencies.

公式总表
Formula sheet

前向 · $O(h)$ / forward

$$f'(x_j) \approx \frac{f_{j+1} - f_j}{h}$$

误差 $-\frac{h}{2}f''$ / error $-\frac{h}{2}f''$

后向 · $O(h)$ / backward

$$f'(x_j) \approx \frac{f_j - f_{j-1}}{h}$$

误差 $+\frac{h}{2}f''$ / error $+\frac{h}{2}f''$

中心 · $O(h^2)$ / central

$$f'(x_j) \approx \frac{f_{j+1} - f_{j-1}}{2h}$$

误差 $-\frac{h^2}{6}f'''$ / error $-\frac{h^2}{6}f'''$

二阶导 · $O(h^2)$ / second derivative

$$f''(x_j) \approx \frac{f_{j+1} - 2f_j + f_{j-1}}{h^2}$$

误差 $-\frac{h^2}{12}f^{(4)}$ / error $-\frac{h^2}{12}f^{(4)}$

Taylor 展开 / Taylor series

$$f(x_{j\pm1}) = f_j \pm hf'_j + \frac{h^2}{2}f''_j \pm \frac{h^3}{6}f'''_j + \frac{h^4}{24}f^{(4)}_j \pm \cdots$$

阶数 / order

$$E \approx Ch^p, \quad p \approx \log_2 \frac{E(h)}{E(h/2)}$$

log-log 斜率 $= p$ / log-log slope $= p$

噪声放大 / noise amplification

$$\epsilon \sin(\omega x) \xrightarrow{\mathrm{d}^k/\mathrm{d}x^k} \epsilon\omega^k$$

噪声–步长权衡 / noise–step trade-off

$$E \le \frac{Mh}{2} + \frac{2\delta}{h}, \quad h^* = 2\sqrt{\delta/M}$$

numpy 写法与数组对齐 / numpy one-liners and array alignment

```
d1f = np.diff(y) / h                    # forward  -> x[:-1]
d1b = np.diff(y) / h                    # backward -> x[1:]
d1c = (y[2:] - y[:-2]) / (2*h)          # central  -> x[1:-1]
d2  = (y[2:] - 2*y[1:-1] + y[:-2]) / h**2 # f''      -> x[1:-1]
g   = np.gradient(y, h)                 # central inside, O(h) ends
```

前向和后向差分是同一个数组 np.diff(y)/h，区别只在配哪组 x：前向配 x[:-1]，后向配 x[1:]。np.gradient 内部用二阶中心差分，端点默认用一阶单侧差分（edge_order=1）。

> Forward and backward differences are the same array np.diff(y)/h; only the paired x differs: forward with x[:-1], backward with x[1:]. np.gradient uses second-order central differences inside and, by default, first-order one-sided differences at the ends (edge_order=1).

考点速记 KEY POINTS

- 单侧 $O(h)$；中心与三点 f'' 为 $O(h^2)$。
  One-sided $O(h)$; central and 3-point f'' are $O(h^2)$.
- 长度：单侧 $N-1$，中心 $N-2$。
  Lengths: one-sided $N-1$, central $N-2$.
- 中心差分分母 $2h$，二阶导分母 h^2。
  Central divides by $2h$; f'' by h^2.
- 有噪声：误差下限 $2\sqrt{M\delta}$。
  With noise: error floor $2\sqrt{M\delta}$.

01 / 6 三种差分 · Three differences

三种差分的数值比较

Comparing the three differences numerically

设 $f(x)=\cos x$，$h=0.1$。分别用前向、后向、中心差分估计 $f'(1)$，与精确值 $-\sin 1$ 比较；再预测 $h=0.05$ 时前向与中心差分的误差。

> **Let $f(x)=\cos x$ and $h=0.1$. Estimate $f'(1)$ with the forward, backward and central differences and compare with the exact $-\sin 1$; then predict the forward and central errors at $h=0.05$.**

答案 Answer 中心最准 · Central best

中心 -0.84007，误差 1.4×10^{-3}，比单侧差分小约 20 倍。

> **Central -0.84007, error 1.4×10^{-3}, about 20× smaller than one-sided.**

计算 / Computation

$$\cos(0.9)=0.621610,\qquad \cos(1)=0.540302,\qquad \cos(1.1)=0.453596,\qquad -\sin(1)=-0.841471$$

方法 Method	近似值 Approx.	误差 近似−精确 · approx − exact	理论首项 · Leading-term prediction
前向 Forward	$(0.453596-0.540302)/0.1=-0.867062$	-0.02559	$\frac{h}{2}f''(1)=-0.02702$
后向 Backward	$(0.540302-0.621610)/0.1=-0.813077$	$+0.02839$	$-\frac{h}{2}f''(1)=+0.02702$
中心 Central	$(0.453596-0.621610)/0.2=-0.840069$	$+0.00140$	$\frac{h^2}{6}f'''(1)=+0.00140$

前向误差 -0.0256 与首项预测 -0.0270 的差 0.0014 正是下一项 $\frac{h^2}{6}f'''(1)=\frac{0.01}{6}\sin 1$。前向与后向误差符号相反，二者平均正好是中心差分。

> The gap between the forward error -0.0256 and the prediction -0.0270 is 0.0014, exactly the next term $\frac{h^2}{6}f'''(1)=\frac{0.01}{6}\sin 1$. The forward and backward errors have opposite signs, and their average is the central difference.

h 减半的预测 / Prediction for halved h

$$h=0.05:\quad E_{\text{fwd}}\approx\tfrac{1}{2}(-0.0256)=-0.0128\quad(\text{actual }-0.01315)$$
$$E_{\text{cen}}\approx\tfrac{1}{4}(0.00140)=0.00035\quad(\text{actual }0.000351)$$

考点速记 KEY POINTS

- 中心差分用 $\cos 0.9$ 与 $\cos 1.1$，除以 0.2。
  Central uses $\cos 0.9$ and $\cos 1.1$, divided by 0.2.
- h 减半：单侧误差 ×1/2，中心 ×1/4。
  Halve h: one-sided ×1/2, central ×1/4.
- 前向、后向误差反号。
  Forward and backward errors have opposite signs.
- 计算器用弧度制！
  Calculator in radians!

02 / 6 精度阶 · Order of accuracy

由误差数据判断方法

Identifying a method from its errors

某方法估计 $f'(x)$ 的误差在 $h = 0.02$ 时为 4.0×10^{-3}，在 $h = 0.01$ 时为 1.0×10^{-3}。它最可能是哪种方法?

A method for $f'(x)$ has error 4.0×10^{-3} at $h = 0.02$ and 1.0×10^{-3} at $h = 0.01$. Which method is it most likely to be?

A 前向差分
Forward difference

B 后向差分
Backward difference

C 中心差分 ✓ 正确 Correct
Central difference

D 无法判断：误差应随 h 线性变化
Cannot tell: the error should change linearly with h

答案 Answer C

h 减半误差变 1/4，$p = 2$，三者中只有中心差分是二阶。

Halving h quarters the error, $p = 2$; only the central difference is second order.

推导 / Derivation

$$p = \log_2 \frac{E(0.02)}{E(0.01)} = \log_2 \frac{4.0 \times 10^{-3}}{1.0 \times 10^{-3}} = \log_2 4 = 2$$

据此还可预测：$h = 0.005$ 时误差约 $1.0 \times 10^{-3}/4 = 2.5 \times 10^{-4}$；常数 $C = E/h^2 = 10$，对应 $\frac{1}{6}|f'''| \approx 10$。

This also predicts the error at $h = 0.005$: about $1.0 \times 10^{-3}/4 = 2.5 \times 10^{-4}$; the constant is $C = E/h^2 = 10$, i.e. $\frac{1}{6}|f'''| \approx 10$.

干扰项为什么错 / Why the distractors are wrong

A 一阶方法 h 减半误差只减半，应为 2.0×10^{-3}。
A first-order method only halves: it would give 2.0×10^{-3}.

B 后向差分同样是一阶，理由同 A。
The backward difference is also first order; same reason as A.

D 只有一阶方法误差才线性变化；两组数据足以测出 p。
Only first-order errors are linear in h; two data points suffice to measure p.

考点速记 KEY POINTS

- $p = \log_2[E(h)/E(h/2)]$。
  $p = \log_2[E(h)/E(h/2)]$.
- 步长比不是 2 时用 $\Delta \log E / \Delta \log h$。
  For other step ratios use $\Delta \log E / \Delta \log h$.
- 比值 2 → 一阶；比值 4 → 二阶。
  Ratio 2 → first order; ratio 4 → second.
- 前提：截断误差主导（h 不太小、无噪声）。
  Assumes truncation dominates (moderate h, no noise).

03 / 6 Python 实现 · Python

中心差分的数组写法

Central difference with numpy arrays

设 h = 0.1，x = np.arange(0, 1, h)（10 个点），y = f(x)。下列哪一组代码正确算出中心差分，并给出与之对齐的 x 坐标?

> **Let h = 0.1, x = np.arange(0, 1, h) (10 points) and y = f(x). Which code computes the central difference correctly, with matching x coordinates?**

A d = (y[2:] – y[:-2]) / (2*h)；xd = x[1:-1] ✓ 正确 Correct

> d = (y[2:] – y[:-2]) / (2*h); xd = x[1:-1]

B d = np.diff(y) / (2*h)；xd = x[1:-1]

> d = np.diff(y) / (2*h); xd = x[1:-1]

C d = (y[1:] – y[:-1]) / h；xd = x[1:-1]

> d = (y[1:] – y[:-1]) / h; xd = x[1:-1]

D d = (y[2:] – y[:-2]) / h；xd = x[1:-1]

> d = (y[2:] – y[:-2]) / h; xd = x[1:-1]

答案 Answer A

y[2:] 是 f_{j+1}，y[:-2] 是 f_{j-1}，对应内点 x[1:-1]，长度都是 8。

> **y[2:] is f_{j+1}, y[:-2] is f_{j-1}, centred on the interior points x[1:-1]; all have length 8.**

推导 / Derivation

对内点 $j = 1, \dots, N-2$：y[2:] 的第 k 个元素是 y_{k+2}，y[:-2] 的第 k 个是 y_k，二者之差除以 $2h$ 就是 x_{k+1} 处的中心差分，正好对应 x[1:-1] 的第 k 个元素。也可直接用 np.gradient(y, h)，它在内点给出同样的结果，并在两端补上单侧差分。

> For interior points $j = 1, \dots, N-2$: element k of y[2:] is y_{k+2} and of y[:-2] is y_k; their difference over $2h$ is the central difference at x_{k+1}, element k of x[1:-1]. np.gradient(y, h) gives the same interior values and fills both ends with one-sided differences.

干扰项为什么错 / Why the distractors are wrong

B np.diff 是相邻点之差（间距 h），除以 $2h$ 得到导数的一半；长度 9 与 x[1:-1] 的 8 也不匹配。

> np.diff takes neighbouring differences (spacing h); dividing by $2h$ gives half the derivative, and length 9 does not match x[1:-1] (8).

C 这是前向差分（长度 9），一阶精度，且与 x[1:-1] 长度不符。

> This is the forward difference (length 9), first order, and mismatched with x[1:-1].

D 漏了因子 2：两点相距 $2h$，结果是导数的 2 倍。

> Missing factor 2: the points are $2h$ apart, so the result is twice the derivative.

考点速记 KEY POINTS

- 中心：(y[2:] – y[:-2])/(2*h) ↔ x[1:-1]。
  Central: (y[2:] – y[:-2])/(2*h) ↔ x[1:-1].
- 漏掉 2 会让结果翻倍。
  Forgetting the 2 doubles the result.
- 长度：$N \to$ 单侧 $N-1 \to$ 中心 $N-2$。
  Lengths: $N \to$ one-sided $N-1 \to$ central $N-2$.
- np.gradient(y, h) 内点即中心差分。
  np.gradient(y, h) is central in the interior.

04 / 6 二阶导数 · Second derivative

三点公式求二阶导数

Second derivative by the three-point formula

用三点公式估计 $f(x)=e^x$ 在 $x=0$ 处的 $f''(0)$，取 $h=0.1$。误差是多少？它与 $\frac{h^2}{12}f^{(4)}$ 还是 $\frac{h^2}{24}f^{(4)}$ 吻合？

> **Use the three-point formula to estimate $f''(0)$ for $f(x)=e^x$ with $h=0.1$. What is the error, and does it match $\frac{h^2}{12}f^{(4)}$ or $\frac{h^2}{24}f^{(4)}$?**

答案 Answer 1.000834

误差 8.34×10^{-4}，与 $h^2/12=8.33\times10^{-4}$ 吻合。

> **Error 8.34×10^{-4}, matching $h^2/12=8.33\times10^{-4}$.**

计算 / Computation

$$f''(0)\approx\frac{e^{0.1}-2e^0+e^{-0.1}}{0.1^2}=\frac{1.105171-2+0.904837}{0.01}=\frac{0.010008}{0.01}=1.000834$$

$$E=1.000834-1=8.34\times10^{-4}$$

$$\text{approx}-f''=\frac{h^2}{12}f^{(4)}(0)=\frac{0.01}{12}\times1=8.33\times10^{-4}\quad\checkmark\qquad\frac{h^2}{24}=4.17\times10^{-4}\neq E$$

再取 $h=0.05$：误差 2.08×10^{-4}，约为原来的 1/4，确认二阶精度。

> With $h=0.05$ the error is 2.08×10^{-4}, about a quarter, confirming second order.

注意 / Caution

幻灯片相加式中的系数写作 $1/24$，本题数值说明应为 $1/12$（见第 6 节编者注）。考试若沿用幻灯片的式子，公式 $f''\approx\frac{f_{j+1}-2f_j+f_{j-1}}{h^2}$ 和"$O(h^2)$"这两个结论都不受影响。

> The slide's summed formula has the coefficient $1/24$; this computation shows it should be $1/12$ (see the Section 6 editorial note). If an exam follows the slide, the formula $f''\approx\frac{f_{j+1}-2f_j+f_{j-1}}{h^2}$ and the "$O(h^2)$" conclusion are unaffected.

考点速记 KEY POINTS

- e^x 的各阶导数在 0 处都等于 1。
  Every derivative of e^x equals 1 at 0.
- 分子相减很小（0.010008），保留足够位数。
  The numerator is tiny (0.010008): keep enough digits.
- 三点公式误差 $\frac{h^2}{12}f^{(4)}$。
  Three-point error $\frac{h^2}{12}f^{(4)}$.
- h 减半，误差 ×1/4。
  Halve h: error ×1/4.

05 / 6 噪声放大 · Noise amplification

噪声在高阶导数中的大小

Noise size in higher derivatives

数据为 $f(x) = \cos x + \epsilon \sin(\omega x)$，$\epsilon = 0.001$，$\omega = 1000$。计算 f、f'、f'' 中噪声的幅度。计算 $f''(x)$ 时，这个噪声可以忽略吗?

> **The data are $f(x) = \cos x + \epsilon \sin(\omega x)$ with $\epsilon = 0.001$ and $\omega = 1000$. Find the noise amplitude in f, f' and f''. Can this noise be ignored when computing $f''(x)$?**

答案 Answer 否 · No

f'' 中噪声幅度 $\epsilon\omega^2 = 1000$，是信号（幅度 1）的 1000 倍。

> **Noise in f'' has amplitude $\epsilon\omega^2 = 1000$, 1000× the signal (amplitude 1).**

推导 / Derivation

$$
\begin{aligned}
f &: \cos x + \epsilon \sin \omega x && \Rightarrow\ \epsilon = 10^{-3}\ (0.1\%) \\
f' &: -\sin x + \epsilon\omega \cos \omega x && \Rightarrow\ \epsilon\omega = 1\ (100\%) \\
f'' &: -\cos x - \epsilon\omega^2 \sin \omega x && \Rightarrow\ \epsilon\omega^2 = 10^3\ (100\,000\%)
\end{aligned}
$$

函数里只有 0.1% 的噪声，一阶导数里已和信号一样大，二阶导数里完全淹没了信号。这就是幻灯片结论"高阶导数对噪声尤其敏感"的定量含义。

> Only 0.1% noise in the function is already as large as the signal in f' and swamps it completely in f''. This is the quantitative meaning of the slide's conclusion that higher-order derivatives are especially sensitive to noise.

考试提醒 / Exam reminder

比较的是**噪声幅度与信号幅度之比**：$\cos x$ 各阶导数的幅度都是 1，噪声的幅度依次乘 ω。要算 f''，必须先滤掉频率 1000 的成分。

> Compare the **ratio of noise to signal amplitude**: every derivative of $\cos x$ has amplitude 1, while the noise gains a factor ω each time. To compute f'', first filter out the frequency-1000 component.

考点速记 KEY POINTS

- 噪声幅度：ϵ，$\epsilon\omega$，$\epsilon\omega^2$。
  Noise amplitude: ϵ, $\epsilon\omega$, $\epsilon\omega^2$.
- 信号 $\cos x$ 各阶导数幅度都为 1。
  Every derivative of $\cos x$ has amplitude 1.
- $\epsilon\omega \geq 1$ 时 f' 已不可信。
  Once $\epsilon\omega \geq 1$, f' is unreliable.
- 先滤波，再求高阶导。
  Filter before higher derivatives.

06 / 6 步长选择 · Choosing h (EXT)

有噪声时步长越小越好吗

Is a smaller step always better with noise?

测量数据噪声 $|e_j| \le \delta = 10^{-4}$，且 $\max|f''| = M = 1$。对含噪数据用前向差分时，减小 h 是否总能提高导数精度？比较 $h = 0.2$、0.02、0.002 的误差上界。

> **Measured data carry noise $|e_j| \le \delta = 10^{-4}$, and $\max|f''| = M = 1$. For the forward difference of noisy data, does reducing h always improve the derivative? Compare the error bounds at $h = 0.2$, 0.02 and 0.002.**

答案 Answer 否 · No

最优 $h^* = 0.02$；h 再缩小 10 倍，误差上界反而增大约 5 倍。

> **The optimum is $h^* = 0.02$; a further 10× smaller h makes the bound about 5× worse.**

推导 / Derivation

$$E(h) \le \frac{Mh}{2} + \frac{2\delta}{h}, \qquad h^* = 2\sqrt{\delta/M} = 2\sqrt{10^{-4}} = 0.02, \qquad E_{\min} = 2\sqrt{M\delta} = 0.02$$

h	截断 $Mh/2$ · truncation	噪声 $2\delta/h$ · noise	上界 · bound
0.2	0.1	0.001	0.101
0.02	0.01	0.01	0.020
0.002	0.001	0.1	0.101

在最优点两项相等。h 偏大时截断误差主导，h 偏小时噪声主导；离最优点 10 倍（无论哪个方向），误差上界都约增大到 5 倍。

> At the optimum the two terms are equal. Too large an h lets truncation dominate, too small an h lets noise dominate; a factor 10 away from the optimum in either direction raises the bound about 5-fold.

注意 / Caution

本题超出幻灯片（标拓展）。幻灯片 5 的"误差随 h 线性减小"针对的是无噪声的精确函数值。

> This problem goes beyond the slides (extension). Slide 5's "error decreases linearly with h" refers to exact, noise-free function values.

考点速记 KEY POINTS

- 含噪数据：h 有最优值，不是越小越好。
  Noisy data: h has an optimum; smaller is not better.
- $h^* = 2\sqrt{\delta/M}$，此时两项相等。
  $h^* = 2\sqrt{\delta/M}$, where both terms are equal.
- 误差下限 $2\sqrt{M\delta}$。
  Error floor $2\sqrt{M\delta}$.
- 与舍入误差的 V 形曲线同理。
  Same V shape as round-off.

答案速查与来源

Answer key and sources

题号 Q	答案 Ans.	要点 Key idea
1	-0.84007	中心差分误差 1.4×10^{-3}，单侧约 0.027 · central error 1.4×10^{-3}, one-sided about 0.027
2	C	$p=\log_2 4=2$ · second order
3	A	(y[2:] – y[:-2])/(2*h) ↔ x[1:-1]
4	1.000834 *	误差 $h^2/12$，幻灯片 /24 应为 /12 · error $h^2/12$; slide's /24 should be /12
5	否 No	f'' 噪声 $\epsilon\omega^2=1000$ · noise in f'' is $\epsilon\omega^2=1000$
6	否 No *	$h^*=0.02$，误差下限 0.02 · $h^*=0.02$, error floor 0.02

带 * 的题涉及幻灯片以外的内容或对幻灯片的更正，见该题页的注意框。

> Starred items involve material beyond the slides or a correction of them; see the caution box on that page.

来源 / Sources

1. PHYB57 C20 lecture slides, "Numerical Differentiation" (PHYB57_C20.pdf, 10 slides)
   用户上传 · uploaded by the user
2. Q. Kong, T. Siauw, A. Bayen, Python Programming and Numerical Methods, Ch. 20 Numerical Differentiation
   https://pythonnumericalmethods.berkeley.edu/notebooks/chapter20.00-Numerical-Differentiation.html
3. Same book, §20.3 Approximating of Higher Order Derivatives (source of the /24 coefficient)
   https://pythonnumericalmethods.berkeley.edu/notebooks/chapter20.03-Approximating-of-Higher-Order-Derivatives.html
4. NumPy documentation, numpy.gradient
   https://numpy.org/doc/stable/reference/generated/numpy.gradient.html
5. SciPy documentation, scipy.signal.savgol_filter
   https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.savgol_filter.html

图均按幻灯片代码用 Matplotlib 重新绘制；所有数值由 Python（numpy 2.4）复算。

> All figures were redrawn with Matplotlib from the slides' code; every number was recomputed in Python (numpy 2.4).

考点速记 KEY POINTS

- 三种 f' 公式 + 三点 f'' 公式必背。
  Memorise the three f' formulas and the 3-point f''.
- 阶数：单侧 1，中心 2，三点 f'' 2。
  Orders: one-sided 1, central 2, 3-point f'' 2.
- 噪声放大因子 ω^k。
  Noise amplification factor ω^k.
- h 太小：舍入或噪声误差占主导。
  Too small h: round-off or noise dominates.